

SkillCircle — Complete Full-Stack AI + PWA Product Build Prompt

Build a complete, production-quality full-stack web application called SkillCircle.

SkillCircle is a student-focused professional networking platform inspired by LinkedIn, where students can create professional profiles, showcase their skills and projects, connect with other students, follow users, create posts, like and comment on posts, discover projects, message other users, receive notifications, and use AI to find relevant people and project opportunities.

The goal is to build a real, deployable product, not a tutorial or static demo.

The application must be mobile-first and installable as a Progressive Web App (PWA).

1. TECHNOLOGY STACK

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

Backend

- Node.js
- Express.js
- TypeScript
- REST API

The backend must be a separate Node.js + Express application. Keep frontend and backend responsibilities clearly separated.

Authentication

Use Firebase Authentication.

Support:

- Email/password signup
- Email/password login
- Google authentication
- Logout
- Forgot password
- Password reset
- Persistent authentication sessions

Important: Firebase handles authentication and passwords. Never store user passwords in MongoDB.

The frontend authenticates using Firebase and sends the Firebase ID token to the backend. The backend verifies the Firebase ID token using Firebase Admin SDK.

2. DATABASE

Use:

- MongoDB
- Mongoose

MongoDB stores:

- User profiles
- Posts
- Likes
- Comments
- Projects
- Followers
- Connections
- Conversations
- Messages
- Notifications

Use proper schemas, relationships, validation, indexes and timestamps.

3. IMAGE STORAGE

Use Cloudinary for:

- Profile pictures
- Cover images
- Post images
- Project images

Image flow:

User selects image → Next.js → Node.js + Express → Cloudinary → Cloudinary URL → MongoDB

Only store Cloudinary URLs and relevant metadata in MongoDB.

Implement:

- File type validation
- File size validation
- Image preview
- Upload progress
- Upload errors
- Mobile camera/gallery support where supported

Optimize large mobile images before uploading when appropriate.

4. AI FEATURES

AI is one of the main differentiating features of SkillCircle.

Create an AI recommendation layer that analyzes:

- Skills
- Interests
- Education
- Projects
- Technologies
- Career/project interests

AI Student Recommendations:

Recommend students based on:

- Complementary skills
- Similar interests
- Project compatibility
- Career interests
- Technology overlap

Show:

- Match score
- Recommended person
- Relevant skills
- Explanation

AI Project Recommendations:

Recommend projects based on:

- User skills
- Interests
- Technologies
- Experience
- Goals

AI Team Matching:

If a project requires React, Node.js, Python and Machine Learning, AI should recommend suitable students and explain why.

Keep the AI provider modular and store its API key in environment variables.

5. PRODUCT GOAL

SkillCircle should answer:

"I'm a student. Who should I connect with, what projects should I work on, and which people have skills that complement mine?"

The platform combines:

- Student profiles
- Professional networking
- Skills
- Projects
- Social posts
- Likes
- Comments
- Followers
- Connections
- Messaging
- Notifications
- AI recommendations

6. MAIN USER FLOW

A new user should be able to:

- 1. Visit SkillCircle**
- 2. Sign up with email/password or Google**
- 3. Complete onboarding**
- 4. Upload a profile picture**
- 5. Add education**
- 6. Add skills**
- 7. Add interests**
- 8. Add projects**
- 9. View their dashboard**
- 10. Discover students**
- 11. Follow students**
- 12. Send connection requests**
- 13. Create posts**
- 14. Add images to posts**
- 15. Like posts**
- 16. Comment on posts**
- 17. Search students**

18. Search projects

19. Receive AI recommendations

20. Create project opportunities

21. Find potential teammates

22. Message connected users

23. Receive notifications

24. Edit their profile

25. Install SkillCircle as a PWA

7. LANDING PAGE

Create a polished startup-style landing page with:

- Navbar
- Hero
- Product explanation
- How SkillCircle works
- AI recommendations
- Student networking
- Project matching
- Features
- CTA
- Footer

Primary CTA: Join SkillCircle.

The design should feel like a modern startup product and not like a generic template.

8. AUTHENTICATION

Login:

- Email
- Password
- Login
- Continue with Google
- Forgot password
- Signup link

Signup:

- Name
- Email

- Password
- Confirm password
- Continue with Google
- Terms acceptance

After signup, redirect to onboarding.

9. PROFILE ONBOARDING

Create a multi-step onboarding flow with a progress indicator.

Step 1 — Basic Information

- Profile picture
- Full name
- Username
- Bio
- Location

Step 2 — Education

- College/university
- Degree
- Branch
- Graduation year

Step 3 — Skills

Allow multiple skills such as React, Next.js, Node.js, Python, Java, UI/UX, Machine Learning and Marketing.

Step 4 — Interests

Examples:

- AI
- Startups
- Web Development
- Design
- Cybersecurity
- Entrepreneurship

Step 5 — Projects

Allow:

- Project name
- Description
- Technologies
- Project image
- GitHub URL
- Project URL

Optional information can be skipped and edited later.

10. DASHBOARD

Create a LinkedIn-inspired but original student networking dashboard.

Desktop:

Sidebar | Main Feed | Recommendations

Sidebar:

- Profile summary
- Profile completion
- Home
- Discover
- Projects
- Messages
- Notifications
- Profile
- Settings

Main feed:

- Create post
- Feed
- Recommended students
- Recommended projects

Right section:

- AI recommendations
- Trending skills
- Suggested connections

11. MOBILE DASHBOARD

Mobile is a first-class experience. Do not simply shrink the desktop layout.

Use a single-column mobile layout:

Header → Create Post → Feed → Feed → Recommendations → Bottom Navigation

Use fixed bottom navigation:

Home | Discover | + | Projects | Profile

Messages and notifications should be easily accessible.

Navigation must:

- Work on iOS and Android
- Respect safe areas

- Have clear active states
- Use touch-friendly buttons
- Remain usable while scrolling

12. USER PROFILES

Every user has a public profile containing:

- Profile picture
- Cover image
- Name
- Username
- Bio
- Location
- College
- Degree
- Graduation year
- Skills
- Interests
- Education
- Projects
- Posts
- Followers
- Following
- Connections

Actions:

- Follow
- Unfollow
- Connect
- Accept
- Reject
- Message
- Remove connection

Own profile: Edit Profile.

Users cannot edit another user's profile.

13. FOLLOW SYSTEM

Implement:

- Follow
- Unfollow

- Followers count
- Following count

Prevent self-follow and duplicate follows. Create notifications when appropriate.

14. CONNECTION SYSTEM

States:

Not Connected → Request Sent → Accepted → Connected

Support:

- Send request
- Accept
- Reject
- Cancel
- Remove connection

Prevent self-connections and duplicate requests. Only connected users can message each other.

15. POSTS

Posts are a core SkillCircle feature.

Each post supports:

- Text
- Optional image
- Author
- Timestamp
- Updated timestamp
- Like count
- Comment count

Users can:

- Create posts
- Edit their own posts
- Delete their own posts
- Like posts
- Unlike posts
- Comment
- Delete their own comments
- View comments

Each feed card should show author profile picture, author name, timestamp, content, image if present, like count, comment count and current user's like state.

16. POST LIKES

Every post must support likes.

Flow:

User clicks Like → POST /api/posts/:postId/like → Authentication middleware → Backend → MongoDB → Updated like state/count → Frontend

Requirements:

- One like per user per post
- Clicking again unlikes
- Prevent duplicate likes
- Display total likes
- Show current user's like state
- Update UI immediately
- Handle API failures

Use optimistic UI where appropriate. If the API fails, revert the optimistic state and show an error.

17. POST COMMENTS

Every post must have comments.

Users can:

- Add comments
- View comments
- Delete their own comments

Each comment displays:

- Profile picture
- Name
- Comment text
- Timestamp

Do not store unlimited comments inside the Post document. Use a separate Comment collection.

API:

GET /api/posts/:postId/comments

POST /api/posts/:postId/comments

DELETE /api/comments/:commentId

Validation:

- Authentication required
- Comment cannot be empty
- Limit comment length
- Users can delete only their own comments

Load comments when the user opens the comment section instead of loading every comment for every post. Use pagination for large comment lists.

18. MOBILE POST EXPERIENCE

Optimize posts for touch screens.

Post actions:

Like | Comment | Share

Like and Comment are required.

Comments should open in a mobile-friendly bottom sheet or dedicated screen. The keyboard must not cover the comment input.

19. POST FEED

Implement:

- Pagination or infinite scrolling
- Loading skeletons
- Empty states
- Error states
- Efficient API requests

Do not load every post at once.

20. PROJECT SYSTEM

Users can create project opportunities.

Project fields:

- Title
- Description
- Technologies
- Required skills
- Image
- GitHub URL
- Live URL
- Owner
- Members

Users can:

- Create
- Edit
- Delete
- View

- Search

AI recommends suitable teammates.

21. SEARCH AND DISCOVERY

Create a Discover page.

Students can be searched by:

- Name
- Username
- College
- Skill
- Interest

Projects can be searched by:

- Project name
- Technology
- Category
- Required skill

Add useful filters.

22. MESSAGING

Users can message connected users.

Features:

- Conversation list
- Start conversation
- Send messages
- Receive messages
- Timestamps
- Read status
- Unread count

Mobile chat must be designed specifically for phones. The keyboard must not cover the input.

If real-time messaging is implemented, use Socket.IO. Otherwise structure the backend so Socket.IO can be added later without major rewrites.

23. NOTIFICATIONS

Notify users when:

- Someone follows them
- Someone sends a connection request

- A connection request is accepted
- Someone likes their post
- Someone comments on their post
- Someone messages them
- AI recommends a connection/project

Include:

- Sender
- Recipient
- Type
- Related entity
- Read/unread
- Timestamp

Display unread notification count.

24. MONGODB MODELS

User:

firebaseUid, name, username, email, profileImage, coverImage, bio, location, college, degree, graduationYear, skills[], interests[], projects[], followers[], following[], connections[], createdAt, updatedAt

Post:

author, content, image, likes[], createdAt, updatedAt

Comment:

post, author, content, createdAt, updatedAt

Project:

owner, title, description, technologies[], image, githubUrl, projectUrl, requiredSkills[], members[], createdAt, updatedAt

ConnectionRequest:

sender, receiver, status, createdAt, updatedAt

Conversation:

participants[], lastMessage, updatedAt

Message:

conversation, sender, receiver, content, read, createdAt

Notification:

recipient, sender, type, relatedId, read, createdAt

Use proper MongoDB indexes for frequently queried fields.

25. BACKEND ARCHITECTURE

Use:

backend/

```
|— src/
| |— controllers/
| |— routes/
| |— models/
| |— middleware/
| |— services/
| |— validators/
| |— utils/
| |— config/
| — server.ts
```

Use Controllers, Services, Routes, Models, Middleware, Validation and Error Handling.

Do not put all business logic into route files.

26. AUTHENTICATION ARCHITECTURE

Authentication flow:

User → Firebase Authentication → Firebase ID Token → Next.js → Express API → Firebase Admin SDK → Verify token → Firebase UID → Find/Create MongoDB User → Authenticated request

Every protected API route must verify the Firebase token.

Never trust user IDs sent from the client.

27. API STRUCTURE

Implement REST APIs such as:

POST /api/users

GET /api/users/:id

PATCH /api/users/:id

GET /api/users/search

POST /api/posts

GET /api/posts

PATCH /api/posts/:id

DELETE /api/posts/:id

POST /api/posts/:id/like

GET /api/posts/:postId/comments

POST /api/posts/:postId/comments

DELETE /api/comments/:commentId

POST /api/follows/:userId

DELETE /api/follows/:userId

POST /api/connections/request

PATCH /api/connections/:id

DELETE /api/connections/:id

GET /api/projects

POST /api/projects

PATCH /api/projects/:id

DELETE /api/projects/:id

GET /api/conversations

POST /api/conversations

GET /api/messages/:conversationId

POST /api/messages

GET /api/notifications

PATCH /api/notifications/:id/read

GET /api/recommendations/users

GET /api/recommendations/projects

Protect appropriate endpoints.

28. PWA

SkillCircle must be a fully installable Progressive Web App.

Implement:

- Web App Manifest
- Service Worker
- App icons
- Maskable icons
- Theme color
- Background color
- Standalone display mode
- Offline fallback
- Appropriate caching
- Install experience

The installed app should open without normal browser UI where supported.

29. PWA MANIFEST

Create a proper manifest containing:

- name
- short_name
- description
- start_url
- display
- background_color
- theme_color
- icons

Provide icons for:

- 192×192
- 512×512

Create appropriate maskable icons and use SkillCircle branding consistently.

30. SERVICE WORKER

Use a Next.js-compatible PWA implementation.

Cache-first:

- Static assets
- App shell resources

Network-first:

- API data
- User-specific content

Do not cache sensitive authenticated API responses insecurely. Do not expose authentication data through the service worker.

31. OFFLINE EXPERIENCE

When offline, show:

"You're offline

Some SkillCircle features need an internet connection. Cached content may still be available."

Do not show a generic browser error.

Cache appropriate non-sensitive resources.

Do not attempt to make AI requests, real-time messaging or authentication requests work offline.

32. PWA INSTALLATION

If the browser supports installation, provide a subtle install prompt:

"Install SkillCircle

Get faster access from your home screen."

Buttons:

- Install
- Not now

Do not repeatedly annoy users. Respect dismissal.

33. MOBILE PERFORMANCE

Optimize for mobile networks and lower-end devices.

Implement:

- Lazy loading
- Image optimization
- Responsive images
- Code splitting
- Skeleton loading
- Efficient API calls
- Pagination
- Infinite scrolling where appropriate
- Optimized Cloudinary images
- Minimal unnecessary JavaScript
- Good Core Web Vitals

34. MOBILE UX

All interactive elements must be touch-friendly.

Pay attention to:

- Bottom navigation
- Forms
- Buttons
- Dropdowns
- Modals
- Comments
- Messaging
- Image upload
- Profile editing
- Search
- AI recommendation cards

Respect mobile safe areas.

Ensure the interface works properly on Android and iOS browsers.

35. UI/UX DESIGN

The product should feel like a modern 2026 startup.

Design principles:

- Minimal
- Professional
- Modern
- Clean typography
- Strong hierarchy
- Generous whitespace
- Subtle borders
- Rounded cards
- Consistent spacing
- Restrained animations
- Responsive
- Accessible

Avoid:

- Generic Bootstrap appearance
- Excessive gradients
- Excessive glassmorphism
- Excessive animations
- Clutter
- Inconsistent spacing
- Random colors

Create a consistent design system.

36. LOADING AND ERROR STATES

Every asynchronous operation must have:

- Loading state
- Success state
- Error state
- Empty state

Use:

- Skeleton loaders
- Button loading indicators

- Toast notifications
- Upload progress
- Network error messages
- Retry options where appropriate

Never leave users staring at a blank screen.

37. SECURITY

Implement production-appropriate security.

Backend:

- Validate requests
- Sanitize inputs
- Verify Firebase tokens
- Protect private routes
- Check resource ownership
- Configure CORS correctly
- Rate-limit sensitive endpoints where appropriate
- Never expose secrets
- Use environment variables

Frontend:

- Never expose private API keys
- Handle authentication state safely
- Never trust client-side authorization

Users can only edit/delete their own posts, comments, projects and profile information unless authorized as an admin.

38. ENVIRONMENT VARIABLES

Create .env.example.

Frontend:

```
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=
NEXT_PUBLIC_API_URL=
```

Backend:

```
PORT=
```

MONGODB_URI=
FIREBASE_PROJECT_ID=
FIREBASE_CLIENT_EMAIL=
FIREBASE_PRIVATE_KEY=
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
AI_API_KEY=

Never hardcode secrets. Never commit .env files to Git.

39. SEED DATA

Create realistic development seed data:

- 15-20 students
- Different colleges
- Different skills
- Different interests
- 10+ posts
- Multiple projects
- Followers
- Connections
- Likes
- Comments
- Notifications

Do not use meaningless placeholder data such as John Doe, Test User or Lorem Ipsum.

40. TESTING

Test all major functionality.

Authentication:

- Email signup
- Email login
- Google login
- Logout
- Password reset
- Protected routes

Profiles:

- Create profile
- Edit profile
- Upload image

- View another profile

Posts:

- Create post
- Edit post
- Delete post
- Like
- Unlike
- Prevent duplicate likes
- Add comment
- Delete own comment
- Prevent unauthorized deletion

Connections:

- Send request
- Accept
- Reject
- Cancel
- Remove
- Prevent duplicate requests

Following:

- Follow
- Unfollow
- Prevent self-follow

Projects:

- Create
- Edit
- Delete
- View
- Search

AI:

- User recommendations
- Project recommendations
- Team recommendations
- Recommendation explanations

Messaging:

- Start conversation
- Send message
- Receive message
- Read status

Notifications:

- Create notification
- Mark as read
- Unread count

PWA:

- Install
- Launch in standalone mode
- Manifest works
- Service worker works
- Offline fallback works
- Mobile navigation works

41. README

Create a complete README containing:

- Product overview
- Features
- Tech stack
- Architecture
- Installation
- Frontend setup
- Backend setup
- MongoDB setup
- Firebase setup
- Cloudinary setup
- AI setup
- PWA setup
- Environment variables
- Running locally
- API overview
- Deployment instructions

42. DEPLOYMENT

The product must be deployment-ready.

Recommended:

Frontend: Next.js → Vercel

Backend: Node.js + Express → Render / Railway / similar

Database: MongoDB Atlas

Authentication: Firebase

Image Storage: Cloudinary

Configure production environment variables correctly.

Verify authentication, API communication, database, image uploads, AI, PWA and CORS after deployment.

43. DEVELOPMENT PHASES

Do NOT generate the entire application blindly in one step.

Build and verify the product in phases.

Phase 1

Project setup + design system + landing page + PWA foundation

Phase 2

Firebase authentication

Phase 3

MongoDB + user profiles

Phase 4

Profile onboarding + Cloudinary image uploads

Phase 5

Posts + likes + comments

Phase 6

Follow + connection system

Phase 7

Project system

Phase 8

Search + discovery

Phase 9

AI recommendations

Phase 10

Messaging

Phase 11

Notifications

Phase 12

Mobile optimization + PWA refinement

Phase 13

Security + validation + error handling

Phase 14

Testing

Phase 15

Production deployment

After every phase:

- 1. Run the application.**
- 2. Test the implemented functionality.**
- 3. Fix errors.**
- 4. Verify existing features still work.**
- 5. Only then move to the next phase.**

44. IMPORTANT AI CODING AGENT RULES

Act as a senior full-stack engineer.

Before implementing anything:

- 1. Inspect the existing project structure.**
- 2. Understand the existing architecture.**
- 3. Reuse existing components and utilities.**
- 4. Do not unnecessarily rewrite working code.**
- 5. Keep frontend and backend responsibilities separate.**
- 6. Use TypeScript throughout.**
- 7. Follow clean coding practices.**
- 8. Do not hardcode secrets.**
- 9. Never store passwords in MongoDB.**
- 10. Never trust client-side authorization.**
- 11. Validate all user input.**
- 12. Handle loading and error states.**
- 13. Maintain consistent UI/UX.**
- 14. Test every major feature.**

- 15. Fix errors instead of hiding them.**
- 16. Never claim a feature works unless it has actually been tested.**
- 17. Avoid unnecessary dependencies.**
- 18. Do not over-engineer simple features.**
- 19. Use real database data instead of fake counters or placeholder functionality.**
- 20. Keep the application scalable and maintainable.**
- 21. Make mobile functionality equal in importance to desktop functionality.**
- 22. Ensure PWA functionality is tested on supported browsers.**
- 23. Preserve existing functionality when adding new features.**

When a requirement is ambiguous, choose the simplest production-appropriate implementation.

Do not add unnecessary features outside the requirements.

FINAL INSTRUCTION

Build SkillCircle as a real full-stack AI-powered student networking product.

The final product must include:

Firebase Authentication + Google Auth + Email Auth + User Profiles + Profile Pictures + Cloudinary + MongoDB + Posts + Post Images + Likes + Comments + Followers + Connections + Projects + Search + AI Recommendations + AI Team Matching + Messaging + Notifications + Mobile-First UI + PWA + Offline Experience + Deployment Readiness.

Architecture:

SkillCircle

|

+-- Next.js Frontend

| |

| +-- PWA Experience

|

+-- Node.js + Express REST API

|

+-- Firebase Authentication

+-- MongoDB Database

+-- Cloudinary Images

+-- AI Recommendation System

Start with Phase 1 only.

Before writing code:

- 1. Inspect the existing project structure.**
- 2. Identify what already exists.**
- 3. Plan the Phase 1 implementation.**
- 4. Implement only Phase 1.**
- 5. Run and verify it.**
- 6. Fix any errors.**
- 7. Explain what was created.**
- 8. Explain which files were changed.**
- 9. Explain how to run it.**
- 10. Explain what needs to be tested.**

Do not proceed to Phase 2 until Phase 1 has been successfully verified.